

Method	Layers/submodules	Neurons	Basis-aligned spaces	Non-basis-aligned spaces
Exhaustive search	$O(\ell)$	$O(\ell \cdot d)$	$O(2^{\ell \cdot d})$	N/A
Gradient attribution	$O(1)$	$O(1)$	N/A*	N/A
Probing	$O(\ell)$	N/A	$O(\ell)$	$O(\ell)$
Alignment search	$O(\ell)$	$O(\ell \cdot d)^\dagger$	$O(\ell)$	$O(\ell)$
Sparse autoencoders	N/A	N/A	N/A	$O(\ell \cdot f)^\ddagger$
Clustering	$O(K)$	$O(K)$	$O(K)$	$O(K)$

Table B.1

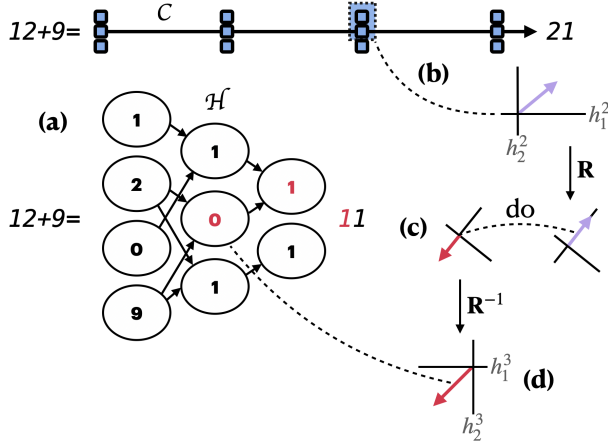
Summary of number of forward passes (and backward passes, when applicable) needed to locate the most causally relevant mediators of a given type (columns) using a particular method (rows). When a method involves training, we do not include training time in these estimates. *One could operationalize this as the sum of neurons' gradient attributions (in which case it would be $O(1)$, though finding the best combination could still be exponential), but this is not recommended for three reasons: interaction effects, redundancy, and potential nonlinear compositions. † This estimate is based on the method of Geiger et al. (2021), but this is not common; with more recent methods like Boundless DAS (Wu et al. 2023), it could in theory be reduced to $O(\ell)$. ‡ This assumes exhaustive search; the time becomes $O(1)$ if using gradient attributions.

Appendix B: Computational Considerations

We have briefly touched on the computational considerations inherent to each mediator type and search method throughout the survey. Here, we expand this discussion by more directly comparing their computational costs.

Table B.1 contains estimates of the number of forward and backward passes through $\mathcal{C}$ needed to locate the most causally relevant mediators, assuming we compute causal relevance using the indirect effect as in Eq. 1. Where applicable, ℓ refers to the number of layers, d to the size of an activation vector $\mathbf{h}^\ell$, f to the size of an SAE's latent vector (i.e., the size of the output of the encoder), and K to the number of clusters (a hyperparameter used in clustering algorithms). We see that gradient attributions are always fastest where applicable, but as they are linear estimates, we expect them to be less accurate than more exact methods like exhaustive search.

Note that training times are excluded from these estimates. In general, training a single probe should have similar amortized runtime compared to a single training run of alignment search, though the difference lies in how many runs would be needed to find the correct features, and in how many examples would be needed to properly train them. If using a typical linear classification probe, one only needs to train $O(\ell)$ times maximum to obtain the best probe. If using Boundless DAS (Wu et al. 2023), one needs to train $O(\ell \cdot |x|)$ times, where $|x|$ is the length of the input sequence. For unsupervised methods, training time can vary significantly; training a sparse autoencoder can take over a day given even a relatively small model of $< 1\text{B}$ parameters (assuming access to one A100 GPU), but one only needs to train $O(\ell)$ of them. For clustering, the training time depends on the clustering method, but one can, in theory, cluster essentially any set of scalars relatively quickly. Moreover, clustering can be performed over all possible components in a model simultaneously without needing to iterate over layers (though it may sometimes be beneficial to perform clustering iteratively).

**Figure C.1**

Example of alignment search, based on an example from Mueller et al. (2025). (a) We start with the computation graph C , and a hypothesized high-level causal graph $\mathcal{H}$. The hypothesis is that the model accomplishes addition using a tens-place addition, a ones-place addition, and a carry-the-one variable. (b) We hypothesize that the carry-the-one variable exists in layer two (h^2). This variable may exist between multiple neurons, so interventions to neurons will not suffice. (c) We learn a rotation R into a new space where the target variable is aligned to the basis. This allows us to perform an intervention (the do -operation) to change the carry-the-one variable to some counterfactual value. (d) After intervening, we rotate back out using R^{-1} . If the hypothesized causal graph is correct, the new output should be 11 instead of 21 after changing the carry-the-one variable's value.

Appendix C: Illustration of Alignment Search

Here, we provide an illustration of an alignment search example (Figure C.1), as described in §6.2, and use this example to illustrate the goals of mechanistic interpretability more broadly. We have a hypothesis as to how the model C performs addition; in other words, we have a guess as to what $\mathcal{H}$ looks like. To isolate a variable in the hypothesized $\mathcal{H}$, we must design a dataset of contrastive pairs that vary only with respect to the variable. For example, if we believe the model contains a carry-the-one feature, we can design a dataset of inputs that vary with respect to whether the model must carry the one while leaving all other variables unchanged. We can use these pairs to isolate the variable during a training procedure; see Wu et al. (2023); Geiger et al. (2024) for more details.

Generally, one does not search for every variable in the hypothesized $\mathcal{H}$; one might select a few variables of interest. To quantify whether one has found them, one performs interventions to the discovered variable, as depicted in Figure C.1. Success is measured by whether the predicted change to the model's output under the intervention given $\mathcal{H}$ is what is actually observed.

Appendix D: Concrete Examples

The discussion in §7.1 is abstract. Here, we aim to give more concrete examples as to when certain kinds of mediators may be more appropriate.

Assume we are interested in understanding how a language model performs multiple-choice question answering. Also assume that we can afford to rerun fine-tuning and adaptation if this model is particularly bad, so we do not intend to perform precise model editing based on the results of benchmarking evaluations or interpretability experiments. Instead, we care mainly about predicting success and failure modes on future examples so that we know whether this model could be deployed in production, and in what cases we should double-check the model's outputs. In this case, the goal is to *explain model behavior*, and we do not have a specific mechanistic hypothesis. Thus, we should deploy an *unsupervised method* to locate and search over meaningful features, such as non-basis-aligned directions. This will help one find unanticipated mechanisms.

Assume instead that we want to precisely edit the knowledge of the model on cases where it gets the answer wrong. Now, we do not necessarily care as much about interpreting the model's general answering process (as helpful as this would be), but rather, debugging and fixing specific mistakes. Thus, this would fall under localization and editing: we would like to locate the source of incorrect answers, and patch them to improve performance. Thus, as a first step, we should deploy an exhaustive search over a relatively coarse-grained mediator, such as submodules or full layers. We can use model editing techniques like ROME (Meng et al. 2022) or MEMIT (Meng et al. 2023), which perform targeted updates to basis-aligned components, to edit facts in cases where the model answered incorrectly. It is theoretically possible that localizing editing over non-basis-aligned mediators could result in even better performance. For example, AlphaEdit (Fang et al. 2025) performs a targeted update to the *null space* of MLP layers, outperforming both ROME and MEMIT on several benchmarks. Future work should investigate whether this is possible, and whether the expected time complexity increase is worth the performance improvements.

Now assume that we are running a different kind of study: the task is still multiple-choice question answering, but we are testing a specific hypothesis as to how the model accomplishes the task. We want to know whether it represents "truthfulness" as an independent concept, and the study is only concerned with to what extent this holds—not accuracy on the task *per se*. Here, we want to verify a specific mechanistic hypothesis, so we should design a dataset of labeled examples, where the label is based on truthfulness, and then deploy a supervised method such as probing over layers. If we can find a way to design counterfactual input pairs that vary only with respect to truthfulness, then we could instead deploy a more precise supervised method such as counterfactual-based optimization over non-basis-aligned subspaces. This would yield a set of scores that indicate to what extent the hypothesis causally explains the model's output behavior.

Note that none of these examples have recommended the use of basis-aligned subspaces, such as (sets of) neurons or attention heads. This is not to say that they are not useful, but it does indicate that when compute is not a significant limitation, they are often not the best place to start when working with realistic neural networks trained on large-scale data. Basis-aligned units are often difficult to interpret, and there are many of them; other mediator types are generally either more interpretable or easier to search over. That said, basis-aligned subspaces may be useful when we expect that they may have interpretable meanings (e.g., in toy task settings), or when we expect that unsupervised methods like sparse autoencoders are likely to yield bad results, or are simply not effectively trainable given one's resources.